What is Formative Evaluation?

Formative evaluation is an ongoing, iterative testing process conducted **during** the design and development phase of a product. Instead of waiting until a website or app is fully coded and launched, designers evaluate early concepts, wireframes, and prototypes with real users.

The Golden Analogy: When the chef tastes the soup while cooking it, that is **formative evaluation**. When the customer tastes the soup at the dinner table, that is **summative evaluation**.

Why Do We Do It? (The Core Benefits)

The primary goal of formative evaluation is to shape and improve the product while it is still easy and cheap to make changes.

- **Catch Usability Issues Early:** It helps identify friction points—like confusing navigation or unclear button labels—before they are hard-coded into the software.
- **Save Development Time and Cost:** Discovering a flawed user flow while testing a paper sketch or a Figma mockup costs almost nothing to fix. Fixing that same flaw after a development team has spent weeks coding it is incredibly expensive.
- **Validate Design Decisions:** It removes the guesswork. Instead of a design team debating which layout is better, they can test both versions (A/B testing) and let the users decide.
- **Iterative Improvement:** It perfectly supports an agile design process. You build a lo-fi prototype, test it (formative evaluation), fix the errors, build a hi-fi prototype, and test it again.

Key Characteristics of Formative Evaluation

- **Qualitative Focus:** It focuses mostly on observational data—asking *why* a user is struggling rather than just measuring *how many* users struggled.
- **Small Sample Sizes:** You only need 5 to 7 users to uncover the majority of major usability flaws in a specific user flow.
- **Conducted on Prototypes:** It relies heavily on the deliverables you just covered: paper wireframes, static mockups, and clickable prototypes.

Common Formative Evaluation Methods

- **Think-Aloud Protocol:** Users are asked to vocalize their thoughts as they try to complete a task on a prototype (e.g., "I'm looking for the checkout button, but I only see the menu icon...").
- **Heuristic Evaluation:** UX experts review the interface against established usability principles (heuristics) to find potential design flaws.

- **Moderated Usability Testing:** A researcher sits with a user, gives them specific tasks to complete on a prototype, and observes their behavior in real-time.

What is Summative Evaluation?

Summative evaluation in UI/UX is the process of evaluating a finished or nearly finished user interface to measure how well it works for users. It is usually done after the design has been developed to determine whether the interface meets its goals and provides a good user experience.

The main purpose is to measure the overall usability and effectiveness of a design rather than to find improvements during the early design stages.

For example, after designing a mobile banking app, you might ask users to perform tasks such as logging in, checking their balance, and transferring money. You can then measure:

Completing the Analogy:

If formative evaluation is the chef tasting the soup while cooking to adjust the seasoning, **summative evaluation** is the customer eating the finished soup at the table and rating it on a review site.

Why Do We Do It? (The Core Objectives)

While formative evaluation is about *improving* the design, summative evaluation is about *measuring* the design's success against established goals.

- **Measure Success Metrics:** It determines if the product met its Key Performance Indicators (KPIs), such as task completion rates, time on task, or conversion rates.
- **Establish a Baseline:** It provides baseline metrics for the current version of the software, which future updates can be measured against.
- **Competitor Benchmarking:** It allows teams to compare their finished product's usability and performance directly against competitor products in the market.
- **Calculate ROI:** It helps business stakeholders understand the return on investment by proving that the new design successfully solved the initial problem.

Key Characteristics & Methods

- **Quantitative Focus:** It relies heavily on hard data, metrics, and statistics (e.g., "85% of users completed the checkout process in under two minutes") rather than observational feedback.
- **Larger Sample Sizes:** Because it requires statistical significance, summative testing typically involves a larger pool of users than formative testing.

- **Conducted on Finished Products:** It is performed on the live software, a fully coded beta version, or a highly advanced interactive prototype.
- **Common Methods:**
    - **Analytics Review:** Tracking real user behavior using tools like Google Analytics or heatmaps.
    - **Standardized Surveys:** Using industry-standard questionnaires like the System Usability Scale (SUS) to get a quantitative usability score.
    - **A/B Testing (Live):** Releasing two different versions of a live page to see which mathematically performs better.

Formative vs. Summative (At a Glance)

Feature	Formative Evaluation	Summative Evaluation
When it happens	During design and development	At the end of development / post-launch
Primary Goal	To identify and fix problems	To measure success and performance
Type of Data	Qualitative (Why is this happening?)	Quantitative (How many, how fast?)
Sample Size	Small (5–7 users)	Large (Requires statistical significance)

What is Usability Testing?

Before diving into the two types, it is helpful to define the core concept for your students: Usability testing is a method of evaluating a product or prototype by testing it with representative users. The goal is to observe how real people interact with the interface, identify where they get stuck, and discover opportunities to improve the overall user experience.

1. Moderated Usability Testing

In a moderated usability test, a researcher (the moderator) is present with the participant during the session. This can happen in person (in a lab or office) or remotely via screen-sharing tools like Zoom or Google Meet.

How it Works:

The moderator guides the user through a set of tasks, asks follow-up questions, and observes both their screen interactions and their body language or facial expressions.

Key Advantages:

- **Deep Qualitative Insights:** The moderator can ask probing questions in real-time (e.g., "I noticed you hesitated before clicking that button, what were you expecting to happen?").
- **Adaptability:** If a user gets completely stuck or a prototype breaks, the moderator can help them recover and keep the test moving.
- **Complex Scenarios:** Best for testing complex, high-fidelity prototypes or entirely new concepts where users might need a bit of initial context.

Disadvantages:

- **High Cost and Time:** It requires scheduling, compensating participants for specific time slots, and dedicating a researcher's time to facilitate every single session.
- **Observer Effect:** Participants might behave differently or try to "please" the researcher because they know they are being watched.

2. Unmoderated Usability Testing

In an unmoderated usability test, the participant completes the test independently on their own device, in their own environment, and on their own time.

How it Works:

Researchers use specialized software platforms (like UserTesting or Maze). The software prompts the user with specific tasks (e.g., "Find a 5-day trekking package and add it to your cart") and records their screen, clicks, and sometimes their webcam and voice as they think aloud.

Key Advantages:

- **Speed and Scalability:** You can send a test link to 50 users in the morning and have all the results and recorded videos back by the afternoon.
- **Natural Environment:** Users are in their normal setting (on their couch, using their own phone), which leads to more natural, authentic behavior.
- **Cost-Effective:** It requires significantly fewer labor hours from the design team since no one has to facilitate the sessions.

Disadvantages:

- **No Real-Time Follow-up:** If a user does something unexpected, you cannot stop the video and ask them *why* they did it.

- **Risk of Confusion:** If a task instruction is poorly written or the prototype has a bug, the user might just give up, and the data for that session is ruined.

Comparison Summary for Students

Feature	Moderated Testing	Unmoderated Testing
Researcher Presence	Present (Active facilitation)	Absent (Automated software)
Best Used For	Complex prototypes, deep qualitative "why" insights	Simple tasks, quick feedback, large-scale quantitative data
Cost & Time	High (Requires scheduling and 1-on-1 time)	Low (Fast, automated, highly scalable)
Flexibility	High (Can ask follow-up questions instantly)	Low (Strictly follows the pre-written script)

Analyzing Test Results: From Raw Data to Action

Once a usability test concludes, the design team is left with a mountain of raw data—video recordings, user quotes, and task completion times. Analyzing this data is the process of finding the signal in the noise to figure out exactly what needs to be fixed before the product ships.

The Step-by-Step Analysis Process

To teach this workflow to your BCA students, break the analysis phase down into clear, sequential steps:

- **Compile the Data:** Gather all observation notes, transcripts, and screen recordings into a single, organized repository.
- **Identify Patterns:** Review the data to find recurring behaviors. If one user struggles, it might be a fluke; if four users fail the exact same task, it is a definitive design flaw.
- **Categorize by Severity:** Group the identified issues by how severely they impact the overall user experience.
- **Extract the "Why":** Shift from simple observation to deeper insight. Instead of merely noting "Users clicked the wrong icon," determine that "The icon metaphor is too abstract and requires a text label."

Prioritizing Gathered Insights

Not all user feedback is equally important, and design teams do not have unlimited time. Insights must be prioritized effectively.

- **Critical Issues:** Roadblocks that completely prevent the user from achieving their primary goal (e.g., a broken checkout button).
- **Minor Issues:** Friction points that annoy the user or slow them down, but do not stop them from completing the task.
- **Cosmetic Issues:** Minor alignment tweaks or subjective aesthetic feedback.
- **Impact vs. Effort Matrix:** A strategic tool used to decide what to fix first, strongly prioritizing high-impact, low-effort changes.

1. Introduction to UX Evaluation

Evaluation is an integral part of the design process. It involves collecting and analyzing data about users' experiences when interacting with a design artifact (like a wireframe, prototype, or finished application).

The Three Main Goals of Evaluation:

1. **Assess System Functionality:** Ensure the system provides the necessary features to allow users to perform their tasks efficiently.
2. **Assess User Experience (UX):** Measure aspects like learnability, usability, user satisfaction, and overall engagement.
3. **Identify Specific Problems:** Detect usability roadblocks, confusing interfaces, or unexpected errors before they reach the general public.

2. The Core Evaluation Paradigms

Evaluation paradigms are the broad frameworks or approaches that guide *how* a design is tested. They generally fall into four main categories:

- **"Quick and Dirty" Evaluation:** Fast, informal feedback gathered from colleagues or available users. It is highly cost-effective and used primarily during the early ideation stages (e.g., showing paper sketches to a coworker).
- **Usability Testing (Controlled Settings):** Users perform specific tasks in a controlled environment (like a usability lab). Evaluators measure quantitative metrics like task completion time, success rates, and the number of errors.
- **Field Studies (Natural Settings):** The evaluator goes out into the user’s actual environment (their office, home, or outdoors) to observe how the system is used naturally, complete with real-world distractions.

- **Predictive Evaluation (Expert Analysis):** Experts use established heuristics and cognitive models to predict user behavior and find flaws without needing to recruit actual users.

3. Evaluation through Expert Analysis

Expert analysis is incredibly practical when a design team lacks the time, budget, or resources to recruit real users. Instead, usability experts evaluate the interface based on established psychological principles and design heuristics.

Key Methods of Expert Analysis:

- **Heuristic Evaluation:** Experts critique the interface design against a set of universally accepted usability principles (heuristics), such as Jakob Nielsen's 10 Usability Heuristics. They rate issues based on severity (from "cosmetic problem" to "usability catastrophe").
- **Cognitive Walkthrough:** Evaluators step through a specific task on the interface and ask four key questions at every step from the user's perspective:
    1. Will the user try to achieve the right effect?
    2. Will the user notice that the correct action is available?
    3. Will the user associate the correct action with the result they are trying to achieve?
    4. If the correct action is performed, will the user see that progress is being made?
- **Model-Based Evaluation:** Experts use theoretical models, such as the **GOMS model** (Goals, Operators, Methods, and Selection rules), to mathematically predict user performance and calculate how long a task will take an expert user to complete.

4. Evaluation through User Participation

While expert analysis is helpful, nothing replaces the insights gained from actual users. User participation techniques involve direct interaction with the target audience.

Key Methods Involving Users:

- **Laboratory Studies:** Users are brought into a controlled, specialist usability laboratory.
    - *Advantage:* Access to specialist equipment (like eye-tracking cameras) and an uninterrupted environment.
    - *Disadvantage:* Lacks the context of the user's natural environment.
- **Think-Aloud Protocol:** Users are asked to verbalize their thought processes as they attempt to complete tasks. This gives evaluators direct access to the user's mental model and confusion points.

- **Observational Techniques:** Evaluators record the user's actions via screen recording, video, or keystroke logging. This helps identify *what* the user did, though follow-up interviews are usually needed to understand *why* they did it.
- **Interviews and Questionnaires:** Asking users directly about their experience using tools like the System Usability Scale (SUS) to generate a quantitative user satisfaction score.

5. Iterative Evaluation: The Continuous Cycle

In modern Human-Computer Interaction (HCI), evaluation is never a one-time activity at the end of a project. It is **iterative**, meaning it happens continuously throughout the design lifecycle.

How Iterative Evaluation Works:

1. **Concept Stage:** Conduct "quick and dirty" evaluations on paper prototypes.
2. **Design Stage:** Perform expert analysis (Heuristic Evaluation) on digital wireframes.
3. **Prototyping Stage:** Conduct Formative Evaluation (Usability Testing) on clickable high-fidelity prototypes.
4. **Implementation Stage:** Conduct Summative Evaluation (Field Studies or Analytics review) on the coded, live software.

The Benefit of Iteration:

By testing repeatedly, teams can iron out minor problems early when changes are cheap and fast. If you only test at the end (the implementation stage), fixing a fundamental architecture problem will require tearing down weeks of expensive development code.

Summary Comparison Table for Students

Feature	**Expert Analysis**	**User Participation**
Who conducts it?	UX professionals and usability experts	Representative end-users
When is it used?	Early to mid-design stages	Mid to late-design stages (Prototypes/Live)
Cost & Time	Generally lower cost, faster	Higher cost, requires recruitment and scheduling
Core Methods	Heuristic Evaluation, Cognitive Walkthrough	Usability Testing, Think-Aloud, Field Studies

Feature	Expert Analysis	User Participation
Main Advantage	Identifies clear structural and standard violations quickly	Reveals unexpected human behaviors and emotional reactions

Here are the detailed lecture notes for the **DECIDE Evaluation Framework** and **Heuristic Evaluation**. These frameworks provide structured, industry-standard methodologies for your students to plan and execute their UX testing.

The DECIDE Evaluation Framework

The DECIDE framework is a foundational tool in Human-Computer Interaction (HCI) that guides researchers and designers through the process of planning, conducting, and analyzing an evaluation. It ensures that testing is not just random, but highly structured and purposeful.
You can teach this to your students as a six-step checklist represented by the acronym **DECIDE**:

- **D - Determine the goals:**
    - What is the overarching purpose of the evaluation?
    - *Example:* Is the goal to figure out why users are abandoning their shopping carts, or to test if a new brand color scheme is appealing?
- **E - Explore the questions:**
    - Break the main goal down into specific, answerable questions.
    - *Example:* "How long does it take a user to find the checkout button?" or "Do users understand the error message when a password fails?"
- **C - Choose the evaluation paradigm and techniques:**
    - Decide *how* you will answer the questions. Will you use a quick-and-dirty method, a formal usability test, a field study, or expert heuristic evaluation?
- **I - Identify the practical issues:**
    - Plan the logistics. Do you need to recruit specific users? Do you need a quiet room, screen recording software (like Figma prototyping tools or OBS), or budget to pay participants?
- **D - Decide how to deal with ethical issues:**

    - Ensure participant safety and privacy. This involves getting informed consent, ensuring user data is anonymized, and making sure participants know they can stop the test at any time.
- **E - Evaluate, interpret, and present the data:**
    - Execute the test, gather the data, analyze it for patterns, and present actionable findings to the design or development team.

Heuristic Evaluation

Heuristic evaluation is one of the most popular forms of **predictive evaluation** (expert analysis). Developed primarily by UX pioneer Jakob Nielsen, it involves having a small group of usability experts examine an interface and judge its compliance with recognized usability principles, known as "heuristics."

How It Works

Instead of bringing in everyday users, 3 to 5 UX professionals individually walk through a digital product. They look for specific violations of UX rules and rate the severity of those problems (e.g., from 0 = "Not a usability problem" to 4 = "Usability catastrophe").

Key Examples of Nielsen's 10 Usability Heuristics

You don't need to cover all ten in exhaustive detail, but highlighting these key heuristics helps students understand what experts look for:

- **Visibility of System Status:** The system should always keep users informed about what is going on.
    - *Example:* A loading spinner or a progress bar when uploading a file.
- **Match Between System and the Real World:** The design should speak the users' language using words, phrases, and concepts familiar to them, rather than internal technical jargon.
    - *Example:* Using a "Shopping Cart" icon instead of a database icon for e-commerce.
- **User Control and Freedom:** Users often perform actions by mistake. They need a clearly marked "emergency exit."
    - *Example:* A prominent "Undo" button after deleting an email.
- **Consistency and Standards:** Users should not have to wonder whether different words, situations, or actions mean the same thing.

    - *Example:* A primary "Submit" button should look identical across every single page of an application.
- **Error Prevention:** Even better than good error messages is a careful design that prevents a problem from occurring in the first place.
    - *Example:* Graying out the "Next" button until all required form fields are correctly filled out.

The Strategic Benefits

- **Speed and Cost:** It is much faster and cheaper than recruiting, scheduling, and paying dozens of end-users for traditional usability testing.
- **Identifies Major Structural Flaws:** Experts can quickly point out obvious architectural or consistency errors that would frustrate actual users later on.
- **Best Used Early:** It is highly effective when used on early digital wireframes before moving on to testing high-fidelity prototypes with real users.

Task Analysis: Breaking Down the User's Journey

What is Task Analysis?

Task analysis is the process of breaking down a user's overarching goal into a sequence of smaller, specific, and discrete steps. It helps designers understand exactly *what* users do, *how* they do it, and the cognitive or physical effort required to complete a goal within a system.

Why Do We Do It? (The Design Value)

- **Identify Friction:** By mapping every single click and thought process, designers can spot unnecessary steps that slow the user down.
- **Inform Information Architecture:** It dictates how menus and screens should be organized. If step 2 always follows step 1, those features should be grouped logically.
- **Design for Reality:** It ensures the software matches how the user actually works, rather than forcing the user to adapt to the software.

Types of Task Analysis

- **Hierarchical Task Analysis (HTA):** Focuses on physical and observable actions. It breaks a high-level goal into sub-tasks.
- **Cognitive Task Analysis (CTA):** Focuses on the mental processes, decision-making, and memory load required at each step.

A Practical Classroom Example

To make this real for your students, outline a **Hierarchical Task Analysis** for booking a trekking package on a site like Nepal View Tours:

- **Main Goal:** Book a 5-day Annapurna Base Camp trek.
- **Sub-Task 1 (Search):** Enter destination -> Select dates -> Click "Search."
- **Sub-Task 2 (Evaluate):** Read itinerary -> Check pricing -> View drone gallery.
- **Sub-Task 3 (Checkout):** Enter traveler details -> Input payment information -> Confirm booking.

If the analysis reveals that users are jumping back and forth between Sub-Task 2 and Sub-Task 3 because the total price isn't clear, the design needs to be fixed.

2. Performance Metrics: Measuring Usability Quantitatively

What are Performance Metrics?

Performance metrics are quantifiable, objective data points used to measure the efficiency and effectiveness of an interface. While qualitative feedback (like the "Think-Aloud" protocol) tells you *why* a user is frustrated, performance metrics give you the hard numbers to prove *how much* it is impacting the system.

Key UX Performance Metrics to Teach

These are the standard metrics evaluated during usability testing:

- **Task Success Rate (Completion Rate):**
    - *What it is:* The percentage of users who successfully complete a specific task from start to finish.
    - *Why it matters:* This is the ultimate baseline. If the success rate for resetting a password is only 60%, the interface is fundamentally failing.
- **Time on Task (Task Duration):**
    - *What it is:* The exact amount of time it takes a user to complete a scenario.
    - *Why it matters:* For data-heavy environments, like an Enterprise Resource Planning (ERP) project such as NepaCore, efficiency is everything. If a new dashboard design reduces the time it takes to process an invoice from 3 minutes to 45 seconds, that is a massive, measurable UX victory.
- **Error Rate:**
    - *What it is:* The number of mistakes a user makes while attempting to complete a task (e.g., entering a date in the wrong format or clicking the wrong navigation tab).
    - *Why it matters:* High error rates indicate that the interface is ambiguous, lacks standard heuristics, or has poor "Error Prevention" mechanics.
- **Learnability:**

- *What it is:* How quickly a user's "Time on Task" and "Error Rate" decrease the second or third time they use the system.
- *Why it matters:* Complex systems might be hard to use on day one, but they should become highly efficient once the user learns the workflow.

Summary Table for Students

Metric	**What it Measures**	**Goal**
Success Rate	Effectiveness (Did they finish?)	100% completion
Time on Task	Efficiency (How fast were they?)	Decrease the total time required
Error Rate	Clarity (Did they make mistakes?)	Minimize unexpected or wrong clicks
Learnability	Onboarding (Does it get easier?)	Faster completion times on repeat uses